**Name –** Pratham Mali | **Div** - TY09 | **Roll no** - 31 | **Subject** - DWM | **Experiment** - 4

**Aim** :- Implementation of Clustering Algorithm (K-means / Agglomerative) using Python

**Introduction** :-

- Clustering groups similar data points. Two common methods are:

1. **K-means**: Partitions data into k clusters by iteratively assigning points to the nearest centroid and updating centroids. Simple but sensitive to initial conditions.
2. **Agglomerative**: Builds a hierarchy of clusters by merging similar ones. Handles various data shapes but can be computationally expensive.

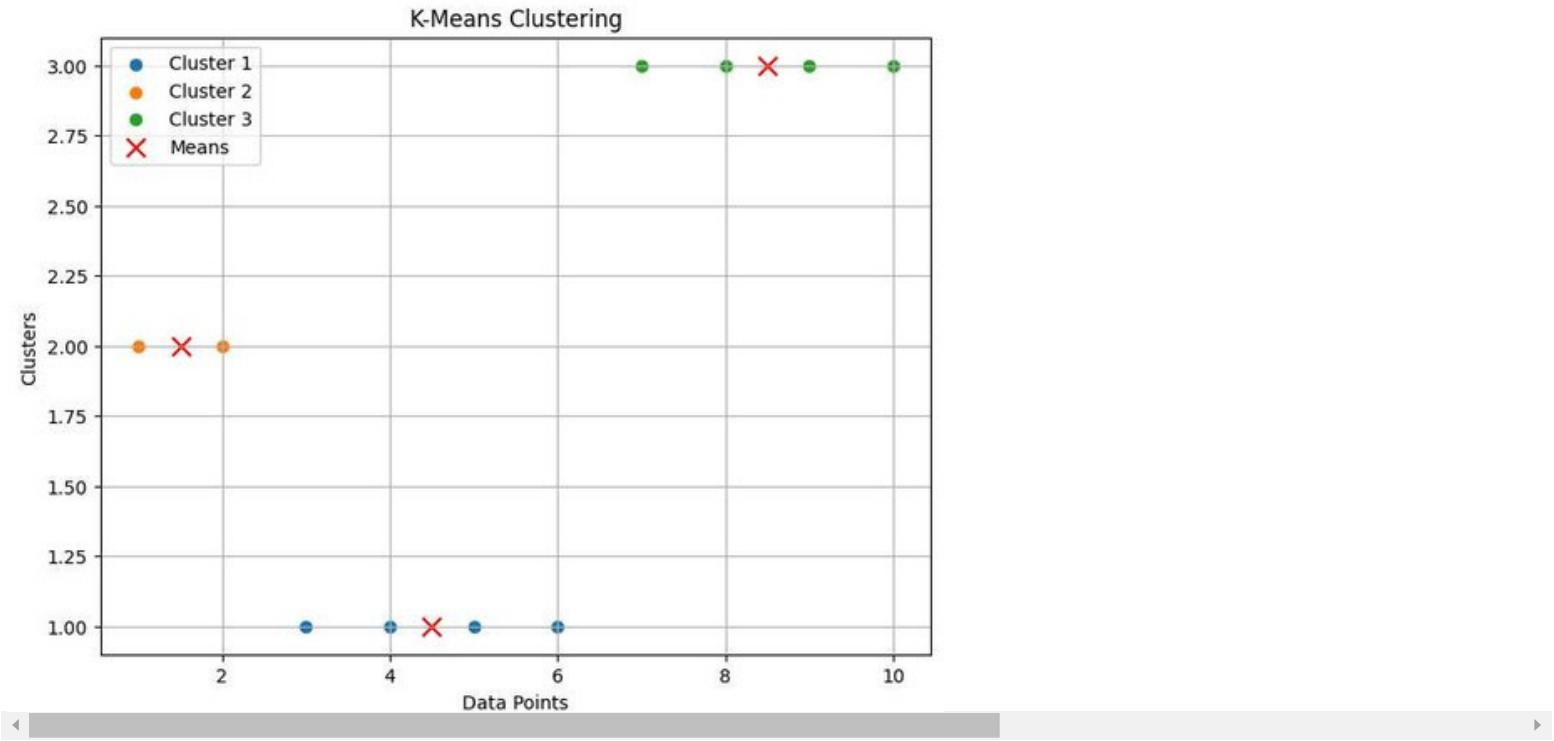
**Procedure** :-

1. Import Necessary Libraries
2. Load and Prepare the Dataset
3. Determine the optimal number of clusters (k)
4. Create and train the KMeans model
5. Get cluster assignments
6. Analyze and visualize the clusters

```
import random
import matplotlib.pyplot as plt

def k_means_clustering():
    # Step 1: Accept user input
    data = list(map(float, input("Enter numbers separated by spaces: ").split()))
    k = int(input("Enter the number of clusters (k): "))

    # Step 2: Initialize cluster means randomly
    means = random.sample(data, k)
    print(f"Initial means: {means}")

    iteration = 0
    while True:
        iteration += 1
        print(f"Iteration {iteration}:")

        # Step 3: Assign each data point to the nearest mean
        clusters = {i: [] for i in range(k)}
        for point in data:
            distances = [abs(point - mean) for mean in means]
            cluster_index = distances.index(min(distances))
            clusters[cluster_index].append(point)

        # Step 4: Calculate new means
        new_means = []
        for i in range(k):
            if clusters[i]:
                new_means.append(sum(clusters[i]) / len(clusters[i]))
            else:
                new_means.append(means[i])  # Keep the same mean if the cluster is empty

        print(f"Clusters: {clusters}")
        print(f"Updated means: {new_means}")

        # Step 5: Check for exact match in means
        if new_means == means:
            print("Exact same means achieved. Clustering complete.")
            break
        means = new_means

    print("Final clusters:")
    for i in range(k):
        print(f"Cluster {i + 1}: {clusters[i]}")

    # Visualization
    plt.figure(figsize=(8, 6))
    for i, cluster in clusters.items():
        plt.scatter(cluster, [i + 1] * len(cluster), label=f'Cluster {i + 1}')
    plt.scatter(means, range(1, k + 1), color='red', marker='x', label='Means', s=100)
    plt.title("K-Means Clustering")
```

```
    plt.xlabel("Data Points")
    plt.ylabel("Clusters")
    plt.legend()
    plt.grid()
    plt.show()

if __name__ == "__main__":
    k_means_clustering()
```

```
Enter numbers separated by spaces: 1 2 3 4 5 6 7 8 9 10
Enter the number of clusters (k): 3
Initial means: [2.0, 1.0, 10.0]
Iteration 1:
Clusters: {0: [2.0, 3.0, 4.0, 5.0, 6.0], 1: [1.0], 2: [7.0, 8.0, 9.0, 10.0]}
Updated means: [4.0, 1.0, 8.5]
Iteration 2:
Clusters: {0: [3.0, 4.0, 5.0, 6.0], 1: [1.0, 2.0], 2: [7.0, 8.0, 9.0, 10.0]}
Updated means: [4.5, 1.5, 8.5]
Iteration 3:
Clusters: {0: [3.0, 4.0, 5.0, 6.0], 1: [1.0, 2.0], 2: [7.0, 8.0, 9.0, 10.0]}
Updated means: [4.5, 1.5, 8.5]
Exact same means achieved. Clustering complete.
Final clusters:
Cluster 1: [3.0, 4.0, 5.0, 6.0]
Cluster 2: [1.0, 2.0]
Cluster 3: [7.0, 8.0, 9.0, 10.0]
```




**Conclusion** :- This experiment demonstrated K-means clustering's effectiveness in partitioning numerical data into distinct, similar clusters. The iterative centroid-based approach revealed underlying data structures, visualized through a scatter plot. While efficient, K-means has limitations with initial conditions and complex data, suggesting further exploration of alternative clustering methods.

**Review Questions** -

1. What is the K-means clustering algorithm, and how does it work?
   Ans :-

- K-means is an unsupervised clustering algorithm that groups data into K clusters based on feature similarity.
- It works as follows

1. Select K random centroids (initial cluster centers).
2. Assign each data point to the nearest centroid. Compute new centroids by averaging the points in each cluster.
3. Repeat steps 2 and 3 until centroids stop changing significantly.

- The final clusters minimize the intra-cluster variance while maximizing the inter-cluster distance.

2. How do you determine the optimal number of clusters in K-means
   Ans -

- Elbow Method – Plot the Within-Cluster Sum of Squares (WCSS) vs. number of clusters and choose the "elbow" point where WCSS stops decreasing significantly.

- Silhouette Score – Measures how well each point fits into its assigned cluster; higher scores indicate better clustering.
- Gap Statistic – Compares WCSS with randomly generated datasets to determine the best K.
- Davies-Bouldin Index – Evaluates the compactness and separation of clusters.

3. What are the common distance metrics used in Agglomerative Clustering?
   Ans -

- Euclidean Distance – Measures straight-line distance between points.
- Manhattan Distance – Measures distance along grid-based paths.
- Cosine Similarity – Measures angle-based similarity (used for text and high-dimensional data).
- Mahalanobis Distance – Accounts for correlations between features.
- Hamming Distance – Used for categorical data by counting differing elements.

**Github Link:-** https://github.com/Pratham510/DWM_TY09_B_31.git